

Lab 5

[valid 2024-2025]

Starting from this week...:

- Pay attention to exception handling (otherwise: -0.5 points)
- Create your own types of exceptions to report abnormal events related to application execution.
- Organize your classes and interfaces in packages.

Input/Output Streams. Files.

Write an application that can manage a collection of images.

Each image has a name, a date, a list of tags, and a location in the file system.

The main specifications of the application are:

Compulsory (1p)

- Create an object-oriented model of the problem. Implement the class describing an image item as a [record](#).
 - Create a class responsible with the operations regarding the repository of images.
 - Implement the following:
 - create an object that represents the repository;
 - add an image into the repository;
 - display the image on the screen (see the [Desktop](#) class).
-

Homework (2p)

- Create a *shell* that allows reading commands from the keyboard, together with their arguments. Represent each command using its own class. Use an interface or an abstract class in order to describe a generic command.
 - Implement the following commands:
 - *add, remove*: individual images in the collection;
 - *update*: the attributes of an existing image;
 - *load, save*: items from/to an external file, either text or binary (using serialization) or JSON. You may use [Jackson](#) or other JSON processing library.
 - *report*: create and open an HTML report representing the content of the repository.
Use a [template engine](#) such as [FreeMarker](#) or [Velocity](#), in order to generate the HTML report.
 - The application will signal invalid data or the commands that are not valid using *custom exceptions*.
 - The final form of the application will be an executable JAR archive. Identify the generated archive and launch the application from the console, using the JAR.
-

Bonus (2p)

- Design a flexible solution that lets the user specify the desired format for persisting the repository: plain text, binary or JSON *or other*.
- Implement the command *addAll*, that adds automatically to the repository all images from a directory and its subdirectories.
- Generate random tags (from a predefined list of your own choice) for each image in your collection.

- Determine all maximal groups of images such that any two images in the group have at least one common tag.
-

Resources

- [Slides](#)
- [Basic I/O](#)
- [Exceptions](#)
- [Regular Expressions](#)
- [Setting the class path](#)
- [Packaging Programs in JAR Files](#)
- [Record Classes](#)

Objectives

- Use basic I/O streams.
- Understand the difference between byte and character streams, primitive and decorator streams.
- Use File I/O to work with files and directories.
- Handle exceptions using *try-catch-finally* blocks.
- Handle resources using *try-with-resources* statements.
- Create custom exceptions and chained exceptions.
- Use regular expressions and [glob patterns](#)
- Understand the notion of CLASSPATH.
- Use external JAR libraries.
- Package all project classes as an executable JAR file.
- Use Java records.